

Solution Requirements

Date	15 July 2026
Team ID	XXXX
Project Name	Emotion Detection Learning Support Engine
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should do).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should be).

Step 1: Functional Requirements (FR)

S. No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	The system does not require user login or identity verification. It is a single-session, open-access tool where a student directly enters text (journal/response) to receive support; no personal credentials are collected.	Low
2	Authorization levels	Single user role only (Student/Learner). No admin panel or differentiated access levels are required since there is no login system; all users interact with the same functionality.	Low
3	External interfaces	Integrates with the Google Gemini API (Generative AI) to generate personalized, empathetic learning-support suggestions based on the detected emotion. No other third-party database or payment API is used.	High
4	Transactions processing	No financial transactions. Core data flow: student text input -> preprocessing (NumPy) -> emotion classification via LSTM/Transformer model (PyTorch) -> emotion label passed as prompt context to Gemini API -> personalized learning-support response returned and displayed in the Streamlit UI.	High
5	Reporting	The system generates an in-session emotion summary (e.g., a simple chart/log of detected emotions over the session) so the student can see their emotional trend while learning. No persistent cross-session analytics dashboard is required.	Medium
6	Business rules, etc.	If negative emotions (e.g., frustration, confusion, anxiety) are detected repeatedly within a session, the system prioritizes supportive, encouraging responses and simplified learning suggestions over purely	Medium

		informational ones.	
7	Compliance to laws or regulations	As the tool may process sensitive student-written text, it follows Responsible AI practices: no permanent storage of personally identifiable information, session data is temporary, and outputs are reviewed for bias/safety per generative-AI responsible-use guidelines.	Medium
8	Other	The system supports Prompt Engineering to tailor Gemini's responses to an educational, age-appropriate, and encouraging tone consistent with a learning-support context.	Low

Step 2: Non-Functional Requirements (NFR)

S. No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	The system should classify the emotion from input text and return a Gemini-generated support response quickly enough to feel conversational.	End-to-end response (model inference + Gemini API call) in under 5 seconds
2	Scalability	As a Streamlit Community Cloud deployment, the app should comfortably handle typical classroom-scale usage without degradation.	Supports up to 50-100 concurrent user sessions
3	Security & Data Privacy	No login credentials are stored. Session-level text/emotion logs are kept only in memory/temporary storage for the duration of the session and are not linked to any identifiable user.	All API calls over HTTPS; no PII persisted beyond session
4	Reliability & Availability	Availability depends on Streamlit Community Cloud and the Gemini API; the app should remain accessible during normal usage with graceful error handling if the API is temporarily unavailable.	Target ~99% uptime during active usage hours
5	Usability & Accessibility	The Streamlit interface should be simple and intuitive for students with no technical background, with clear text input and readable emotion/response output.	Usable by a first-time student user within 1 minute, no training required
6	Other	Maintainability: the codebase is modularized into data preprocessing, model (LSTM/Transformer) inference, Gemini prompt handling, and Streamlit UI layers for easy updates and portability across OS (Windows/Linux/macOS).	Modular Python codebase; runs on Python 3.8+